

TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG TP HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

CÔNG NGHỆ JAVA

GIẢNG VIÊN: THS. ĐẶNG ĐỨC TRUNG

Mục tiêu học phần

- ▶ Kiến thức:
 - ▶ Hiểu, phân tích, thiết kế và vận dụng được công nghệ JavaSE
 - ▶ Hiểu, phân tích, thiết kế và vận dụng được công nghệ JavaEE, Spring.
- ▶ Kỹ năng:
 - ▶ Xây dựng được ứng dụng máy tính dùng JavaSE.
 - ▶ Xây dựng được ứng dụng web dùng JavaEE, Spring.
- ▶ Năng lực tự chủ – trách nhiệm:
 - ▶ Rèn luyện đức tính tỉ mỉ, cẩn thận trong công việc
 - ▶ Có tác phong đĩnh đạc, tự tin, có trách nhiệm trong công tác

Đánh giá học phần

- ▶ **Điểm đánh giá quá trình:**
 - ▶ Trọng số: 40%, bao gồm: 20% chuyên cần, 30% kết quả bài tập thực hành và 50% bài thi giữa kỳ.
 - ▶ Hình thức thi giữa kỳ: Trắc nghiệm
- ▶ **Điểm thi kết thúc học phần: 60%**
 - ▶ Hình thức thi: nộp tiểu luận và dự án

Phương pháp dạy và học

- ▶ Trình bày lý thuyết kết hợp trao đổi hỏi đáp và thảo luận nhóm.
- ▶ Thực hiện các bài tập để minh họa, củng cố kiến thức lý thuyết
- ▶ Giờ thực hành sinh viên thực hành trực tiếp trên máy theo sự hướng dẫn của giảng viên.
- ▶ Thực hành trên dự án thực tế.

Chương trình

CHƯƠNG 1: Công nghệ J2SE và lập trình ứng dụng Desktop

CHƯƠNG 2: Công nghệ J2EE và lập trình Servlet

CHƯƠNG 3: Công nghệ Java Server Page (JSP)

CHƯƠNG 4: Lập trình ứng dụng web với Spring MVC

CHƯƠNG 1. Công nghệ J2SE và lập trình ứng dụng Desktop

- 1.1. Giới thiệu công nghệ J2SE
- 1.2. Lập trình giao diện
- 1.3. Kết nối cơ sở dữ liệu

1.1. Giới thiệu công nghệ J2SE

- ▶ J2SE (Java 2 Platform, Standard Edition) là **nền tảng chuẩn của Java** do Sun Microsystems (nay thuộc Oracle) phát triển, được dùng để xây dựng và chạy các ứng dụng Java trên máy tính để bàn (desktop) hoặc các ứng dụng độc lập (standalone applications).
- ▶ **J2SE** là một trong ba phiên bản chính của Java 2 Platform (bên cạnh J2EE và J2ME).
- ▶ J2SE hiện nay được đổi tên thành **Java SE (Java Standard Edition)**

Các thành phần chính của Java SE

- ▶ J2SE cung cấp một bộ thư viện và công cụ lập trình quan trọng:
- ▶ JVM (Java Virtual Machine): Máy ảo Java để chạy chương trình.
- ▶ JDK (Java Development Kit): Bộ công cụ cho lập trình viên (compiler, debugger, tools).
- ▶ API (Application Programming Interface): các thư viện các hàm đa dạng, phong phú.

Các thư viện chính

- ▶ `java.lang`, `java.util`: xử lý chuỗi, bộ sưu tập, toán học
- ▶ `java.io`, `java.nio`: nhập xuất dữ liệu, xử lý file
- ▶ `java.net`: lập trình mạng
- ▶ `java.swing`, `java.awt`: lập trình giao diện đồ họa (GUI)
- ▶ `java.sql`: kết nối và thao tác với cơ sở dữ liệu

Ứng dụng của Java SE

- ▶ Viết ứng dụng **desktop** (VD: phần mềm quản lý, ứng dụng GUI)
- ▶ Viết ứng dụng dòng lệnh (**console applications**)
- ▶ Phát triển ứng dụng mạng cơ bản (chat, FTP client/server)
- ▶ Nền tảng để học và sử dụng các công nghệ Java khác như **J2EE (Java Enterprise Edition)** và **J2ME (Java Micro Edition)**

1.2. LẬP TRÌNH GIAO DIỆN

- ▶ Giới thiệu thư viện AWT
- ▶ Giới thiệu thư viện Swing
- ▶ Giới thiệu JavaFX

Giới thiệu thư viện AWT

- ▶ AWT viết tắt của **Abstract Windowing Toolkit**
- ▶ AWT là tập hợp các lớp Java cho phép chúng ta tạo một GUI
- ▶ Cung cấp các mục khác nhau để tạo hoạt động và hiệu ứng GUI như
 - ▶ Containers
 - ▶ Components
 - ▶ Layout managers
 - ▶ Graphics và drawing capabilities
 - ▶ Fonts
 - ▶ Events

Giới thiệu thư viện Swing

- ▶ Thư viện **Swing** là một phần của **Java Foundation Classes (JFC)**, được dùng để xây dựng **giao diện người dùng đồ họa (GUI)** cho ứng dụng Java chạy trên desktop. Nó được giới thiệu từ **Java 1.2** (năm 1998) và là bước tiến lớn so với thư viện **AWT**.
- ▶ **Một số thành phần phổ biến:**
 - ▶ **JFrame** – cửa sổ chính của ứng dụng
 - ▶ **JPanel** – vùng chứa các thành phần UI khác
 - ▶ **JButton** – nút bấm
 - ▶ **JMenuBar, JMenu, JMenuItem** – menu ứng dụng
 - ▶ **JLabel** – hiển thị văn bản hoặc hình ảnh
 - ▶ **TextField / JTextArea** – nhập văn bản
 - ▶ **JTable** – bảng dữ liệu
 - ▶ **JTree** – cây thư mục.

Giới thiệu JavaFX

- ▶ **JavaFX** là một **framework** trong Java dùng để xây dựng **ứng dụng giao diện đồ họa (GUI)** hiện đại.
- ▶ JavaFX thay thế dần **Swing** và **AWT** trong Java, được thiết kế để tạo UI trực quan, đẹp mắt, hỗ trợ **CSS**, **FXML**, **đa phương tiện**, và **animation**.
- ▶ Ưu điểm:
 - ▶ Có UI hiện đại, có thể kết hợp FXML với CSS.
 - ▶ Dễ thiết kế giao diện bằng công cụ Scene Builder.
 - ▶ Hỗ trợ đa nền tảng.

Các thành phần chính của JavaFX

1. Stage: đại diện cho cửa sổ ứng dụng.
2. Scene: đại diện cho cảnh (màn hình) chứa các thành phần giao diện. Có thể có nhiều Scene trong một Stage có khả năng chuyển đổi qua lại.
3. Node: mọi thành phần giao diện trong JavaFX đều là node. Các node được tổ chức theo cây.
4. Layout Pan: các thùng chứa (container) để bố trí các thành phần giao diện.
5. FXML: là ngôn ngữ XML dùng để tách giao diện ra khỏi code java.
6. CSS: mã định dạng cho các thành phần giao diện.
7. Media & Animation: hỗ trợ phát nhạc, video, hiệu ứng, diễn hoạt.

Tải và cài JavaFX

- ▶ Vào trang chính thức: <https://gluonhq.com/products/javafx/>
- ▶ Chọn phiên bản phù hợp với JDK bạn đang dùng (ví dụ JDK 17 → JavaFX 17).
- ▶ Chọn gói **SDK** cho hệ điều hành của bạn (Windows / macOS / Linux).
- ▶ Giải nén thư mục JavaFX vào một vị trí cố định (ví dụ C:\javafx-sdk-17.0.2).

Cấu hình JavaFX trong Eclipse

- ▶ Bước 1 – Tạo project Java
 - ▶ Mở Eclipse -> File -> New -> Java Project
- ▶ Bước 2 – Thêm thư viện JavaFX vào project
 - ▶ Chuột phải vào project → **Properties**
 - ▶ Chọn **Java Build Path** → tab **Libraries** → **Modulepath** → **Add External JARs...**
 - ▶ Vào thư mục JavaFX vừa giải nén → chọn **tất cả file .jar trong thư mục lib**
 - ▶ Nhấn **Apply and Close**

Cấu hình JavaFX trong Eclipse

- ▶ Bước 3 – Thêm VM Arguments để chạy
 - ▶ JavaFX cần chỉ định module path và modules khi chạy
 - ▶ Chuột phải file chứa “**main**” → **Run As** → **Run Configurations....**
 - ▶ Trong tab **Arguments** → mục **VM arguments**, thêm vào chuỗi sau:

```
--module-path "C:\javafx-sdk-17.0.2\lib" --add-modules  
javafx.controls,javafx.fxml
```

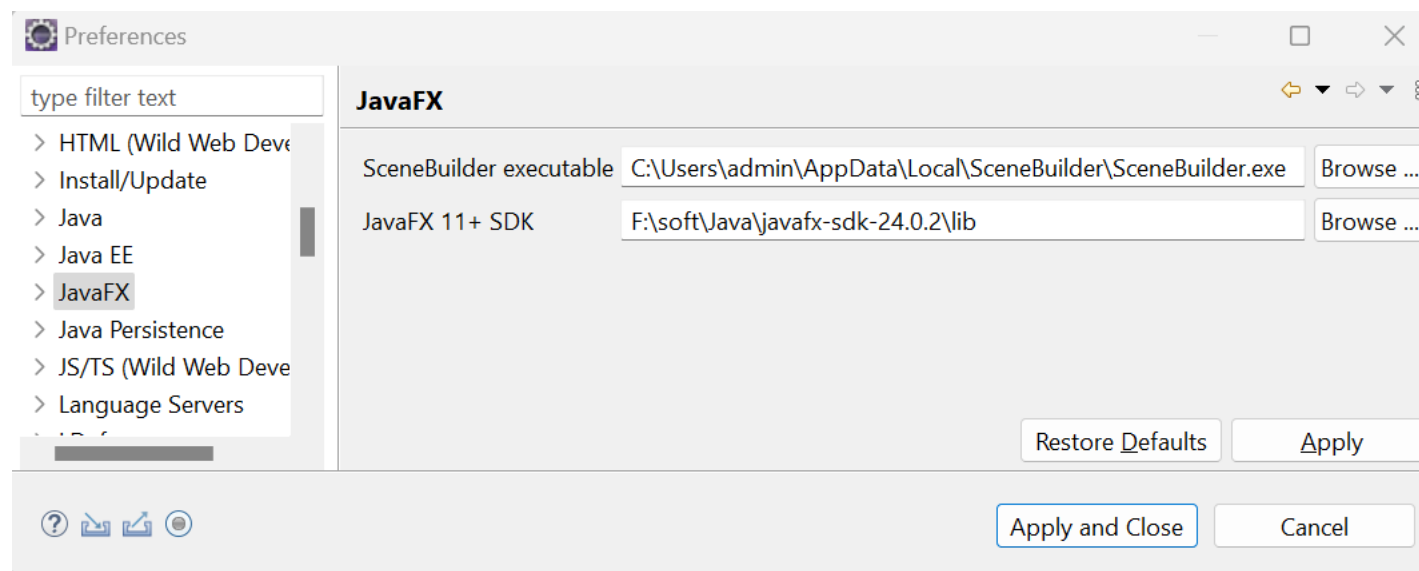
(Thay đường dẫn bằng nơi bạn giải nén JavaFX)
- ▶ Bước 4 - trên eclipse chọn Help -> Installs new software, dán link vào ô Work with, chọn các e(fx)clipse. (<https://download.eclipse.org/efxclipse/updates-released/3.9.0/site/>)

Trình thiết kế giao diện kéo-thả

- ▶ Cho phép bạn **kéo-thả các thành phần UI** (Button, Label, TableView,...) để thiết kế giao diện, sau đó lưu dưới dạng **FXML**.
- ▶ Vào trang tải: <https://gluonhq.com/products/scene-builder/>
- ▶ Tải bản phù hợp với hệ điều hành (Windows / macOS / Linux).
- ▶ Cài đặt như phần mềm bình thường.
- ▶ Nếu muốn tích hợp vào Eclipse:
 - ▶ Mở **Window** → **Preferences** → **JavaFX**.
 - ▶ Trong mục **Scene Builder executable**, trỏ đến file SceneBuilder.exe

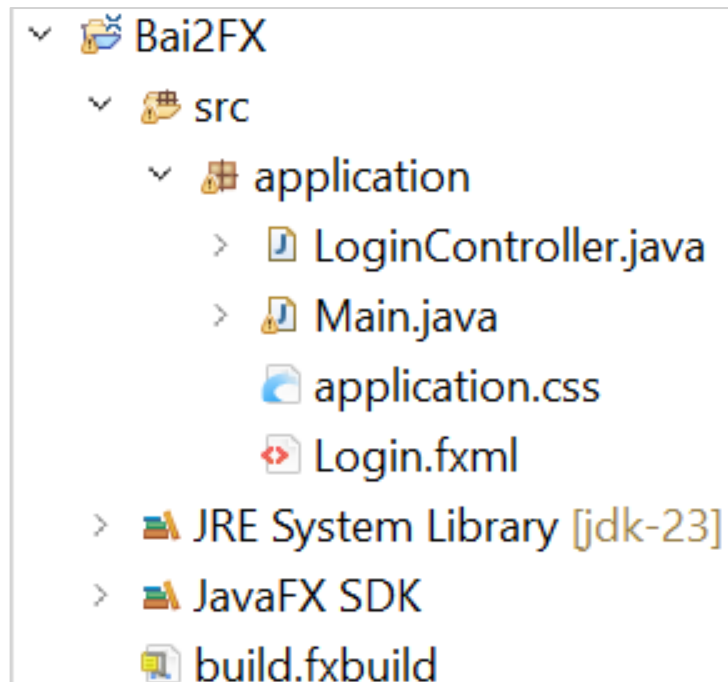
Kết nối JavaFX với Scene builder

- ▶ Vào Eclipse → **Window** → **Preferences** → **JavaFX**
- ▶ Ở mục **Scene Builder executable**, chọn đường dẫn đến file SceneBuilder.exe
- ▶ Ở mục JavaFX 11 + SDK, chọn đường dẫn thư mục lib chứa các file thư viện f(x)
- ▶ Nhấp chọn nút Apply and Close



Tạo project JavaFX

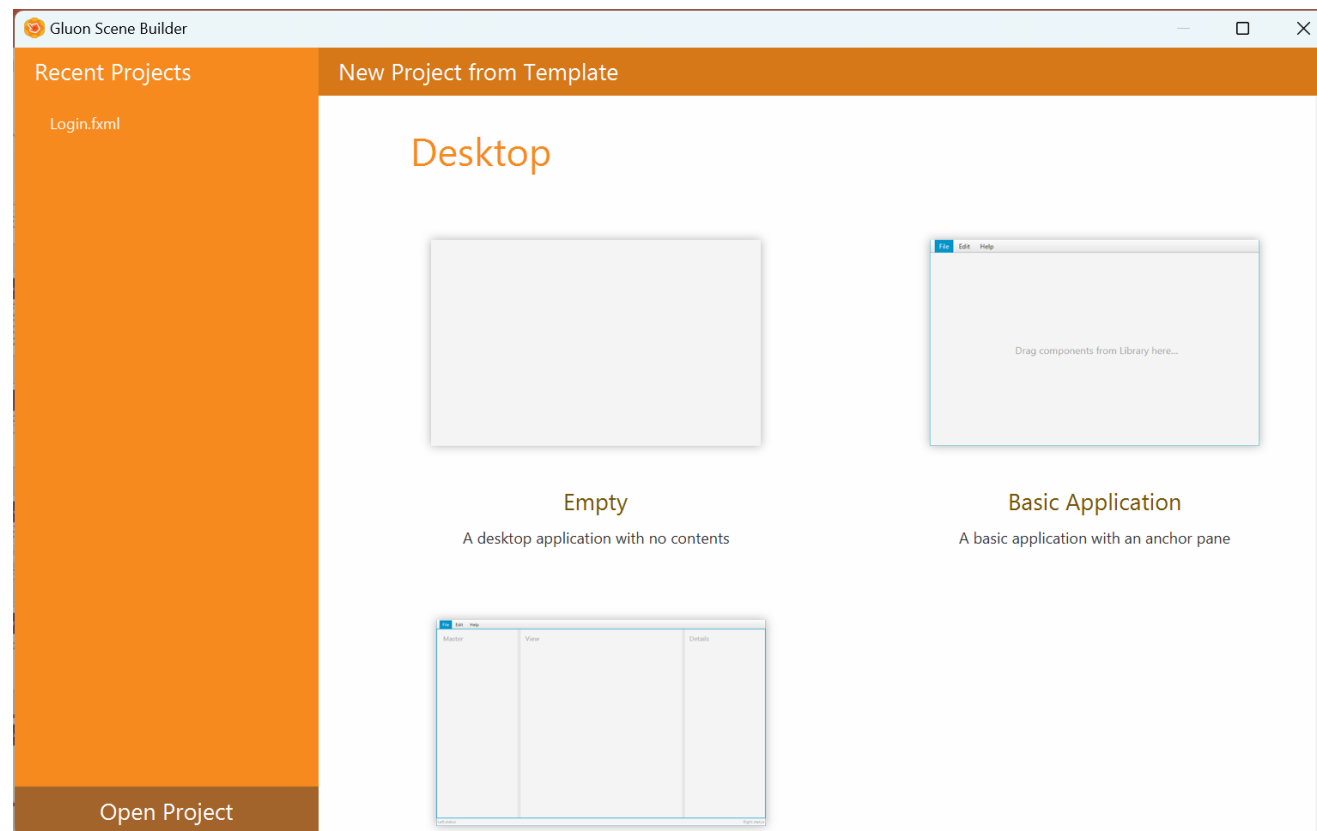
- ▶ File -> New ->Project
- ▶ Mở rộng mục JavaFX, chọn JavaFX project
- ▶ Nhấp Next, đặt Project name, nhấp Finish



Cấu trúc của một Project JavaFX
phát triển trên Eclipse

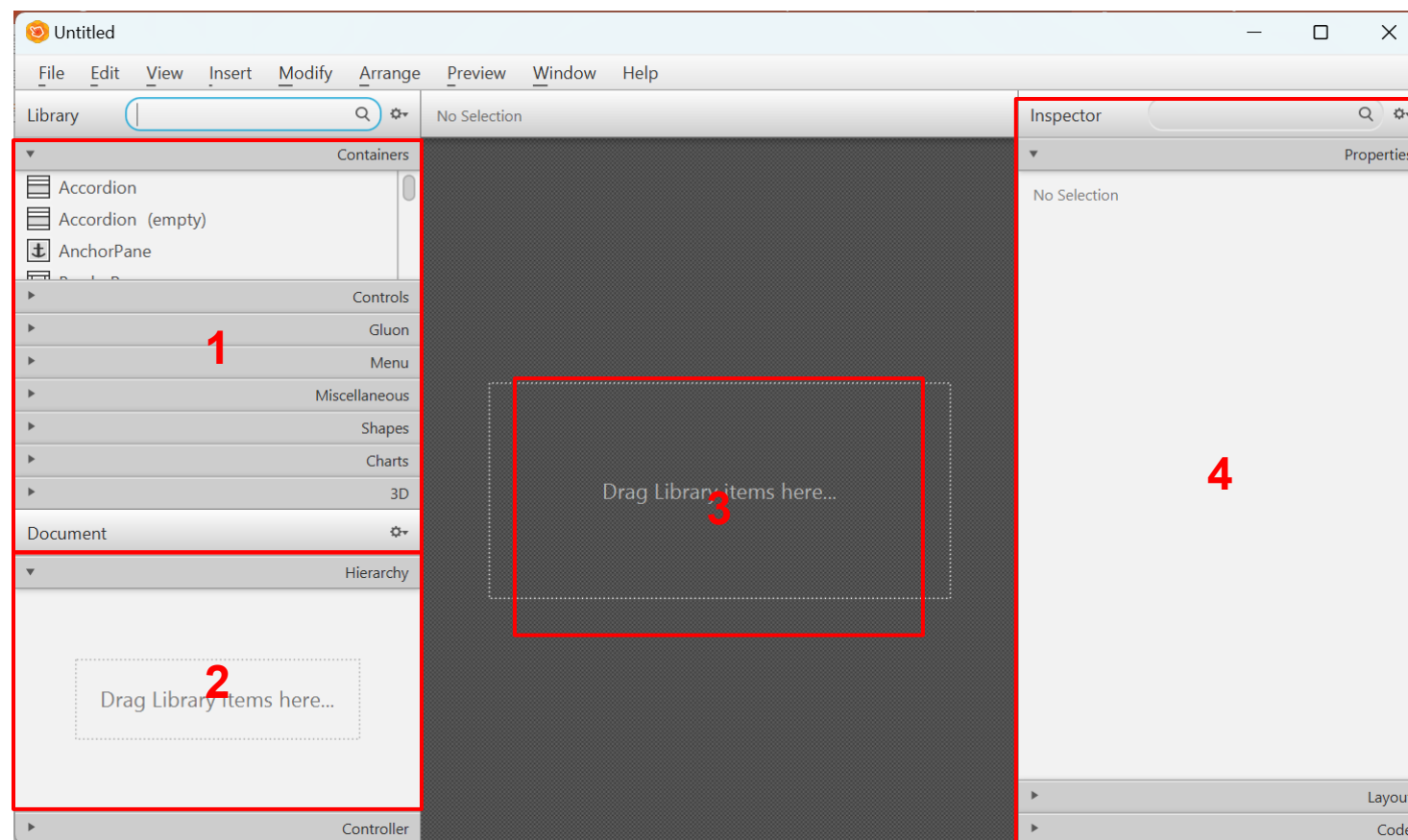
Thiết kế giao diện ứng dụng bằng Scene Builder

- ▶ Khởi động Scene Builder từ hệ thống
- ▶ Chọn nhóm ứng dụng Desktop hoặc Mobile
- ▶ Chọn một kiểu khởi đầu ứng dụng (Empty, Basic app, ...)

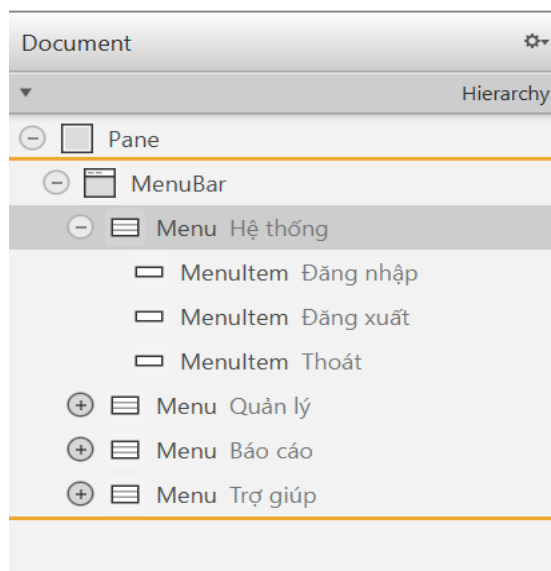


Thiết kế giao diện ứng dụng bằng Scene Builder

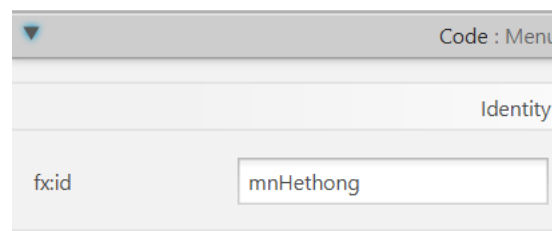
1. Các thành phần giao diện kéo thả, được cung cấp bởi Scene Builder
2. Cấu trúc các thành phần trong giao diện đang thiết kế
3. Vùng thiết kế giao diện
4. Các thông tin thuộc tính, bố cục và kết nối mã nguồn của các thành phần giao diện



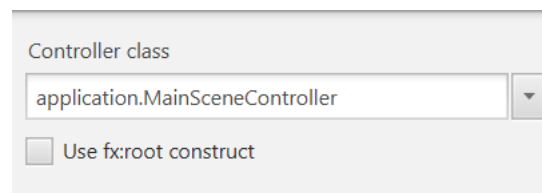
Ví dụ sử dụng các điều khiển



Một số menu



Đặt ID cho một menu



Chỉ định lớp điều khiển



Giao diện

Ví dụ về một lớp điều khiển giao diện

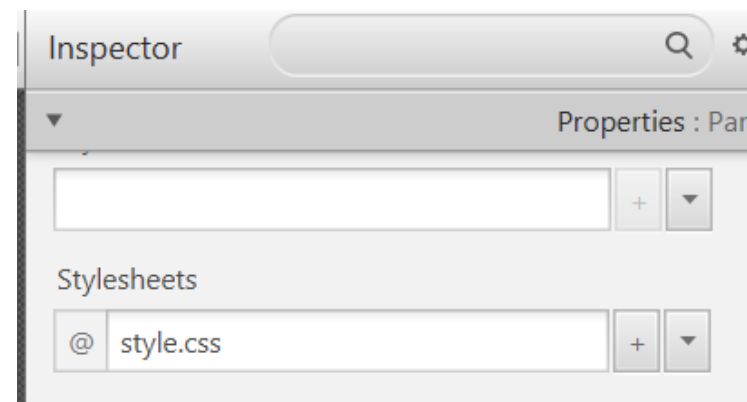
```
package application;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.stage.Stage;
public class MainSceneController {
    @FXML
    private MenuItem mnDangnhap;
    ...
}
```

```
@FXML
private void handleDangnhap() {
    try {
        FXMLLoader loader = new
        FXMLLoader(getClass().getResource("Login.fxml"));
        Parent root = loader.load();
        // Tao Stage (cửa sổ) mới
        Stage stage = new Stage();
        stage.setTitle("Đăng nhập");
        stage.setScene(new Scene(root));
        stage.show();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Định dạng giao diện bằng CSS

```
.root{  
  -fx-background-color: green;  
}  
#mnBar{  
  -fx-background-color: green;  
  -fx-background-color: #10b981;  
}
```

File style.css



Chỉ định file css cho Scene

1.3. Kết nối cơ sở dữ liệu

1.3.1. Giới thiệu JDBC

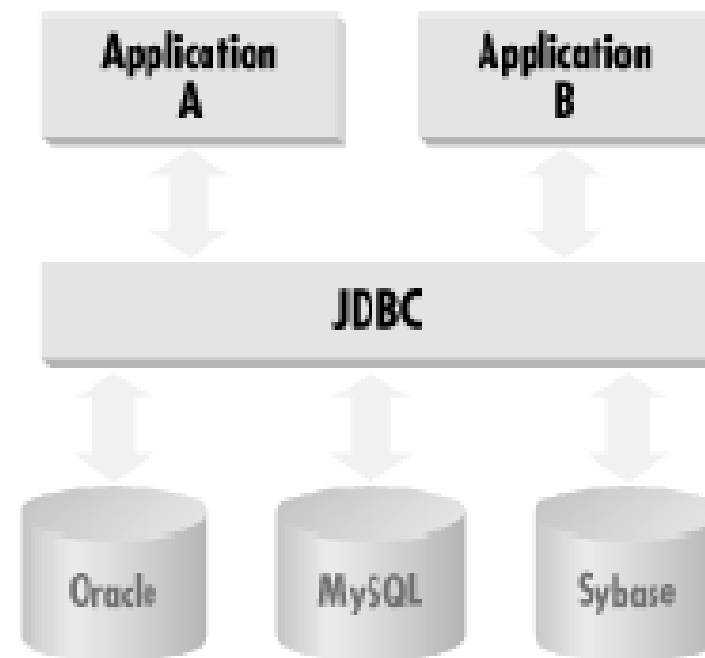
1.3.2. Kiến trúc JDBC

1.3.1. Giới thiệu

- ▶ Hầu hết các chương trình máy tính đều có truy xuất thông tin trong các cơ sở dữ liệu.
- ▶ JDBC API là thư viện chứa các lớp và giao diện hỗ trợ lập trình viên Java kết nối và truy cập đến các hệ cơ sở dữ liệu.
- ▶ `java.sql`: chứa các lớp và giao diện cơ sở của JDBC API.
- ▶ `javax.sql`: chứa các lớp và giao diện mở rộng.

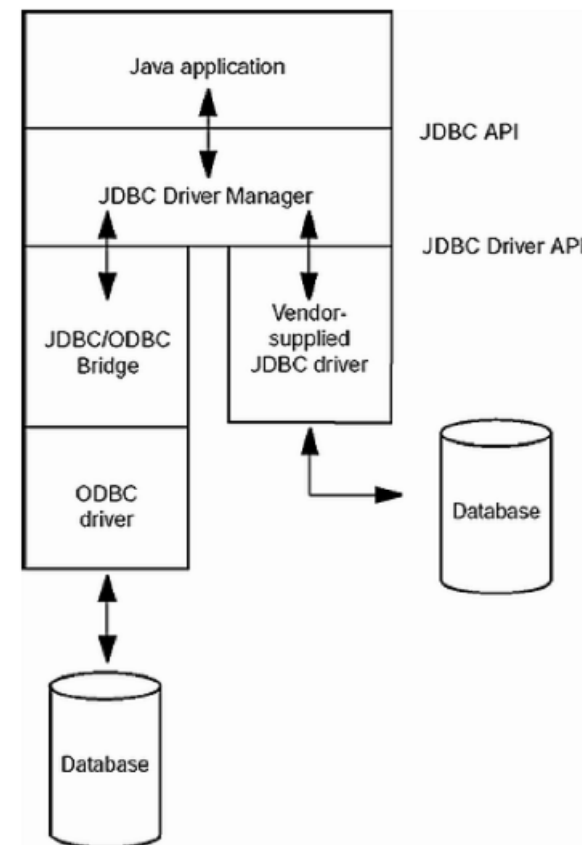
1.3.1. Giới thiệu (tt)

- ▶ JDBC cung cấp tập các lớp và interface cho phép chương trình Java có thể nói chuyện được với hệ CSDL
- ▶ Tập các lớp của JDBC có thể làm việc được với mọi hệ csdl.



1.3.2. Kiến trúc JDBC

- ▶ JDBC gồm 2 tầng:
 - ▶ Tầng 1: là các JDBC API, có nhiệm vụ chuyển các câu lệnh SQL cho bộ quản lý trình điều khiển JDBC;
 - ▶ Tầng thứ 2 là các JDBC Driver API, thực hiện nhiệm vụ liên hệ với trình điều khiển của hệ quản trị cơ sở dữ liệu cụ thể.

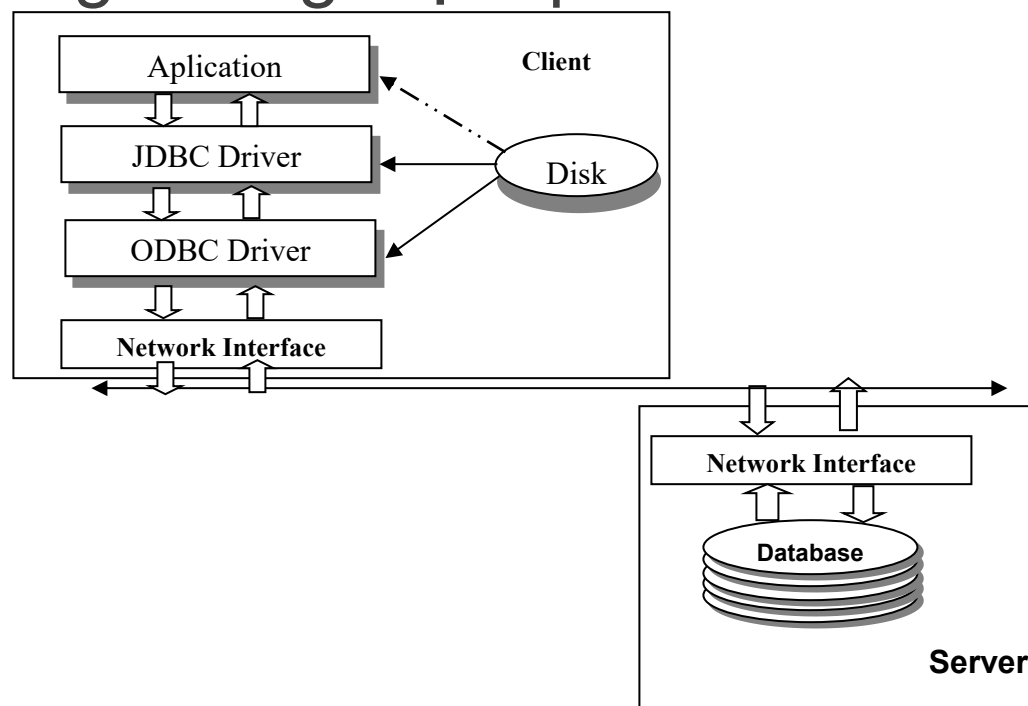


1.3.3. JDBC Driver

- ▶ Dùng để truy cập đến các hệ quản trị cơ sở dữ liệu.
- ▶ Có khả năng yêu cầu hệ quản trị CSDL cụ thể thực hiện các câu lệnh SQL.
- ▶ Có 4 loại JDBC Driver
 - ▶ Loại 1: JDBC/ODBC
 - ▶ Loại 2: Native-API
 - ▶ Loại 3: Open Protocol-Net
 - ▶ Loại 4: Proprietary-Protocol-Net
- ▶ Loại 2,3,4 nói chung được viết bởi nhà cung cấp csdl. hiệu quả hơn loại 1 nhưng thực hiện phức tạp hơn.

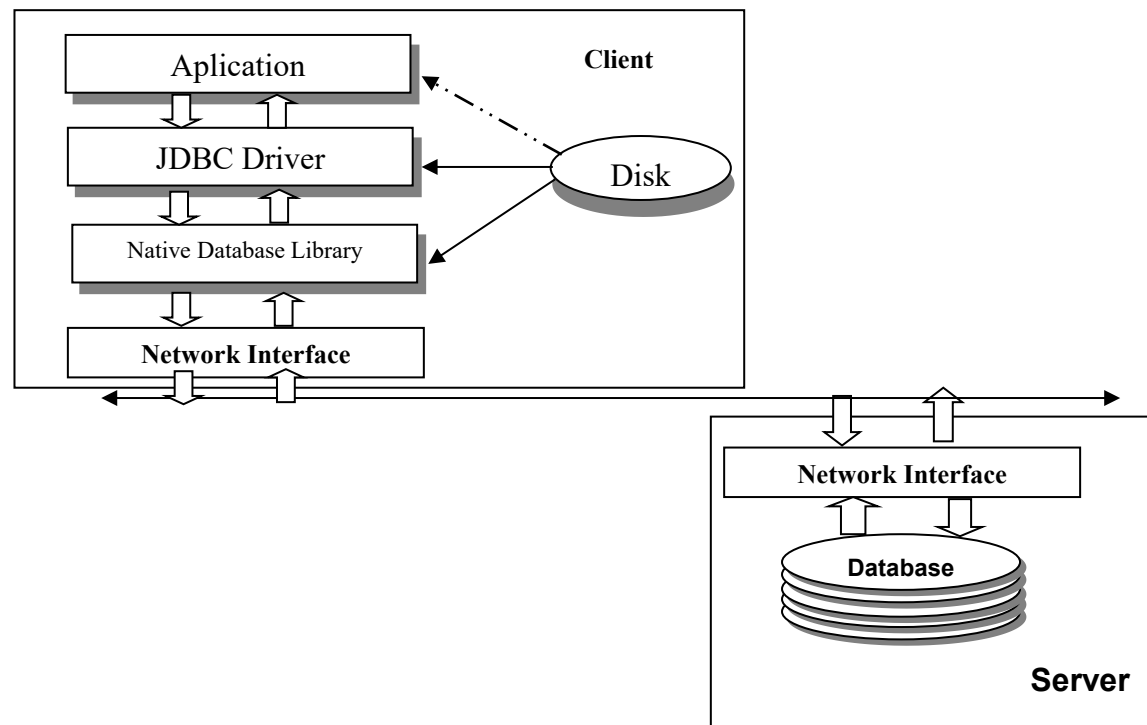
Loại I. JDBC/ODBC

- ▶ jdk hỗ trợ cầu nối jdbc-odbc (jdbc-odbc bridge).
- ▶ Mềm dẻo nhưng không hiệu quả.



Loại 2: Native-API

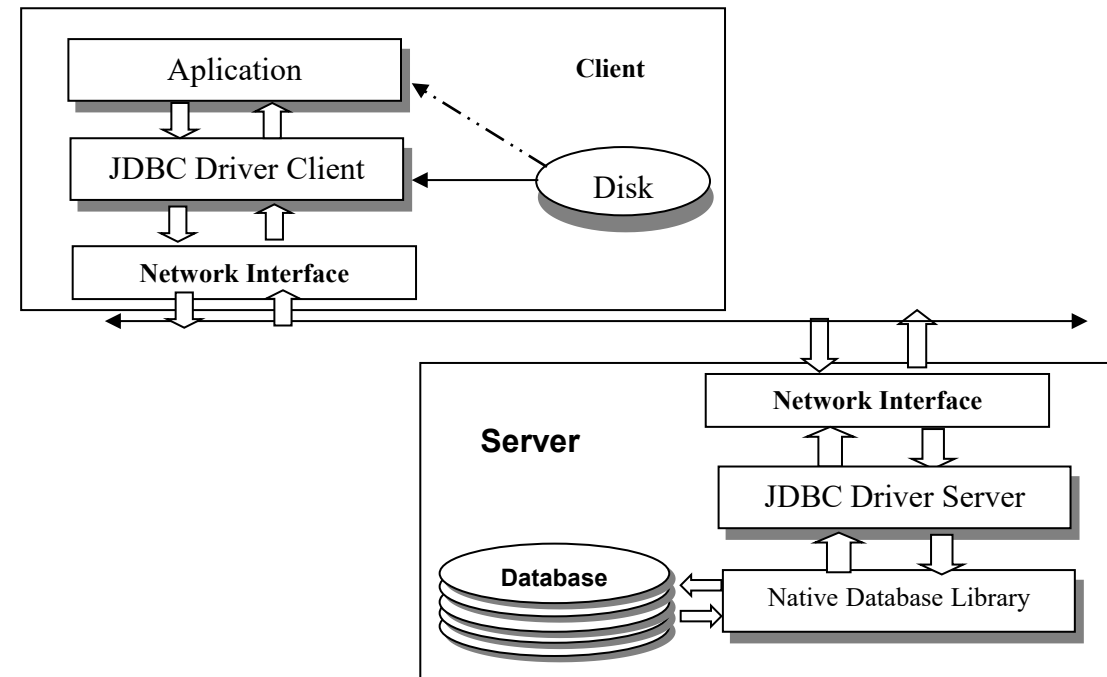
- ▶ Tốt hơn loại 1, loại này cho phép JDBC giao tiếp trực tiếp với các driver hay các hàm API của CSDL.



Loại 3: Open Protocol-Net

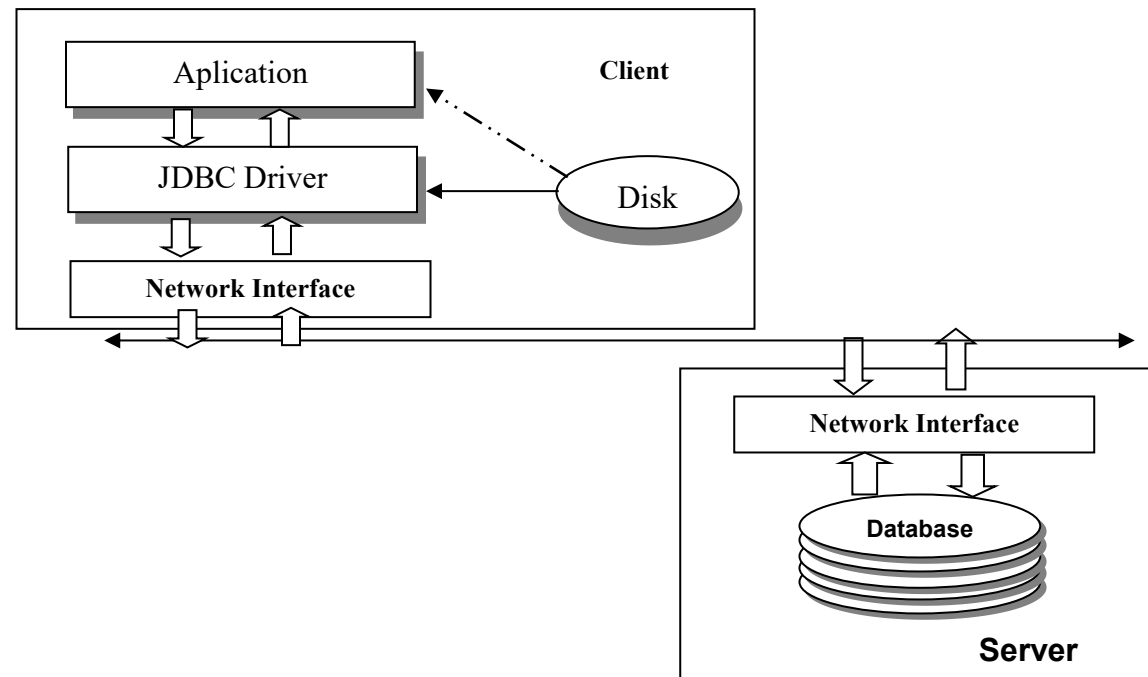
► Drivers

- Có thể chuyển các yêu cầu đến các csdl nằm ở xa.
- Có thể giao tiếp với nhiều loại CSDL.
- Không phải của nhà cung cấp csdl
- Tất cả bằng mã java



Loại 4: Proprietary-Protocol Net

- ▶ 100% java
- ▶ Có khả năng giao tiếp trực tiếp với hệ CSDL không cần chuyển đổi



JDBC URL

Để có thể kết nối với CSDL, chúng ta cần xác định nguồn dữ liệu cùng với các thông số liên quan dưới dạng 1 URL như sau:

jdbc:<subprotocol>:<dsn>:<others>

- ▶ ***<subprotocol>***: được dùng để xác định trình điều khiển để kết nối với CSDL.
- ▶ ***<dsn>***: địa chỉ CSDL. Cú pháp của ***<dsn>*** phụ thuộc vào từng trình điều khiển cụ thể.
- ▶ ***<other>***: các tham số khác

1.3.4.KẾT NỐI CSDL VỚI JDBC

- ▶ Có 3 bước chính để kết nối CSDL.
 1. Nạp database drivers
 2. Tạo nối kết, Tạo đối tượng Connection
 3. Tạo đối tượng Statement để thực thi các lệnh sql..

Ví dụ

```
try{  
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");  
  
    Connection con= DriverManager.getConnection("jdbc:odbc:ATM");  
  
    Statement stmt = con.createStatement();  
    :  
    :  
    :  
    :  
    :
```

Database URL

- ▶ Database URL là một chuỗi được dùng để kết nối csdl.
- ▶ cú pháp : `jdbc:subprotocol name:other_stuff`
- ▶ The subprotocol name được dùng tùy vào loại driver sử dụng để kết nối csdl.
- ▶ ví dụ : subprotocol name là `odbc` nếu driver là cầu nối `jdbcodbc`
- ▶ `Other_stuff` cũng phụ thuộc vào loại driver nào được sử dụng. ví dụ nếu driver là cầu nối `jdbcodbc` thì thành phần này là tên của đối tượng ODBC

Database Driver

- Bảo đảm ứng dụng java tương tác với mọi csdl dưới một cách thức chuẩn và duy nhất.
- Bảo đảm những yêu cầu từ chương trình sẽ được biểu diễn trong csdl dưới một ngôn ngữ mà csdl hiểu được
- nhận các yêu cầu từ client, chuyển nó vào định dạng mà csdl có thể hiểu được và thể hiện trong csdl.
- Nhận các phản hồi, chuyển nó ngược lại định dạng dữ liệu java và thể hiện trong ứng dụng.

Nạp Driver

- Lớp DriverManager chịu trách nhiệm nạp driver và tạo kết nối đến csdl.

`DriverManager.registerDriver(new sun.jdbc.odbc.JdbcOdbcDriver());`

- hoặc

Class.forName(String);

- This returns the object associated with the class with the given string name.
- `Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");`
- Equivalent to:
`new sun.jdbc.odbc.JdbcOdbcDriver();`
If you have a driver from another vendor, then find out the class name of that driver and load it instead.

Gói Java.sql

- ▶ Cung cấp tập hợp các lớp và interface dùng để trao đổi với CSDL.
- ▶ Các lớp
 - ▶ DriverManager
 - ▶ Date, Time
 - ▶ Timestamp
 - ▶ Types
- ▶ Các Interfaces
 - Driver
 - Connection
 - DatabaseMetaData
 - Statement
 - PreparedStatement
 - CallableStatement
 - ResultSet
 - ResultSetMetaData

Đối tượng Statement

43

- ▶ Đối tượng Connection chứa liên kết trực tiếp đến csdl.
- ▶ Sử dụng đối tượng Connection để tạo đối tượng Statement.
 - ▶ `Statement s = con.createStatement();`
- ▶ Đối tượng này có nhiệm vụ gửi các câu lệnh sql đến csdl.
- ▶ **`executeQuery(String)`** or **`executeUpdate(String)`** method
- ▶ Cùng một đối tượng Statement có thể sử dụng cho nhiều câu lệnh sql khác nhau.

Đối tượng Statement (tt)

- ▶ Có 3 phương thức thực thi
 - ▶ `executeQuery()`
 - ▶ `executeUpdate()`
 - ▶ `execute()`
- ▶ The `executeQuery()`
 - ▶ Nhận câu lệnh SQL (select) làm đối số, trả lại đối tượng `ResultSet`
- ▶ `ResultSet rs = s.executeQuery("SELECT * FROM Books");`

Đối tượng Statement (tt)

- ▶ Phương thức `executeUpdate()`
 - ▶ Nhận các câu lệnh sql dạng cập nhật
 - ▶ Trả lại số nguyên biểu thị số hàng được cập nhật.
 - ▶ UPDATE, INSERT, or DELETE.
- ▶ Phương thức `execute()`
 - ▶ Được áp dụng cho trường hợp không rõ loại sql nào được thực hiện.
 - ▶ Được áp dụng cho trường hợp câu lệnh sql được tạo ra tự động bởi chương trình.

- Chứa một hoặc nhiều hàng dữ liệu từ việc thực hiện câu lệnh truy vấn.
- Có thể lấy dữ liệu từng hàng dữ liệu một trong ResultSet.
- Sử dụng phương thức next() để di chuyển đến hàng dữ liệu tiếp theo trong ResultSet.
- Hàm next() trả lại true chỉ rằng hàng chứa dữ liệu, trả lại false hàng cuối không chứa dữ liệu.

➤ Thực hiện

```
while (rs.next()){  
    // examine a row from the results  
}
```

- ▶ Để lấy dữ liệu ở các cột trên mỗi hàng của ResultSet, ta dùng các phương thức.
 - ▶ `getType(int | String)`
 - ▶ Đối số là chỉ số cột tính từ 1.
 - ▶ Áp dụng cho các cột có kiểu dữ liệu là int, float, Date.....
 - ▶ Ví dụ :
 - ▶ `String isbn = rs.getString(1); // Column 1`
 - ▶ `float price = rs.getDouble("Price");`

ResultSet Metadata

- Đối tượng này cho biết thông tin về ResultSet
- ***ResultSet rs = stmt.executeQuery(SQLString);
ResultSetMetaData rsmd = rs.getMetaData();
int numberOfColumns = rsmd.getColumnCount();***
- ***getColumnName(int column)***